

MODULE 2

Relational model concepts, Constraints

Relational Algebra: Unary, Binary and Set theory relational operations

Data definition commands: attribute constraints, SET operations, Aggregate functions, Null Values, Nested sub queries, complex queries, Views Data control commands

Data manipulation commands: Insert, Update, Joined relations

Integrity and security: Domain constraints, Referential integrity, Triggers; Security and Authorization in SQL

Relational Algebra

- ☆ It is a **procedural** query language.
- ☆ RA is a **set of operations** on relation(s).
- ☆ It is a set of **algebraic operations**.
- ☆ Input : One or more relations.
- ☆ Output : A relation.
- ☆ It provides a **theoretical foundation** for relational databases.
- ☆ **Structured Query Language (SQL)**.
- ☆ It allows us to understand database operations in more detail and motivate us to write **optimized queries**.



Relational Algebra Operations

Fundamental operations:

- ☆ Select (σ)
- ☆ Project (Π)
- ☆ Union ($\cup$)
- ☆ Set Difference ($-$)
- ☆ Cartesian Product ($\times$)
- ☆ Rename (ρ)

Relational Algebra Operations

Additional operations:

- ☆ Set intersection ($\cap$)
- ☆ Assignment operation ($=$)
- ☆ Natural join ($\bowtie_*$)
- ☆ Division operation ($\div$)
- ☆ Outer join
 - Left outer join ($\bowtie_{\text{L}}$)
 - Right outer join ($\bowtie_{\text{R}}$)
 - Full outer join ($\bowtie_{\text{F}}$)

1. Select (σ)

- ☆ **Select tuples** that satisfy a given predicate.
- ☆ It is denoted by lowercase Greek letter **sigma** (σ).
- ☆ Syntax: $\sigma_{\langle \text{selection_condition} \rangle}(\text{Relation})$.
- ☆ Example: $\sigma_{D_ID = 2}(\text{Employee})$.
- ☆ Comparison operators: $=, \neq, <, \leq, >, \text{ and } \geq$.

⌘ Pull up for precise seeking

1. Select (σ)

Example 1: Write an RA expression to find all the instructors working in "Finance" department.

Solution:

$\sigma_{\text{Dept_Name} = \text{"Finance"}}(\text{INSTRUCTOR})$

Output:

ID	Name	Dept_Name	Salary
26589	Yusuf	Finance	95000
12547	Neil	Finance	80000

INSTRUCTOR

ID	Name	Dept_Name	Salary
10101	John	Biology	65000
12121	Robin	Computer Science	90000
25252	Alya	Electrical	40000
26589	Yusuf	Finance	95000
54789	Ravi	Music	60000
78787	Raj	Physics	87000
87458	Jayant	History	75000
76985	Pratik	Computer Science	89000
12547	Neil	Finance	80000

1. Select (σ)

Example 3: Find all instructors who are working in "Finance" department and drawing the salary greater than \$87,000.

Solution:

$\sigma_{\text{Dept_Name} = \text{"Finance"} \wedge \text{Salary} > 87000}$ (INSTRUCTOR)

Output:

ID	Name	Dept_Name	Salary
26589	Yusuf	Finance	95000

INSTRUCTOR			
ID	Name	Dept_Name	Salary
10101	John	Biology	65000
12121	Robin	Computer Science	90000
25252	Alya	Electrical	40000
26589	Yusuf	Finance	95000
54789	Ravi	Music	60000
78787	Raj	Physics	87000
87458	Jayant	History	75000
76985	Pratik	Computer Science	89000
12547	Neil	Finance	80000

2. Project (Π)

- ☆ It returns its argument relation with **certain attributes left out**.
 - ☆ It is a **unary** operator.
 - ☆ It is denoted by the uppercase Greek letter **pi** (Π).
 - ☆ Basically a relation is a **set**.
 - ☆ In the result, the **duplicate rows** are eliminated.
 - ☆ Syntax: $\Pi_{\text{Attribute1, Attribute2, \dots}}(\text{Relation})$.
-

2. Project (Π)

Example 1: List all instructors' ID, name, and salary, but do not care about the dept_name.

Solution:

$\Pi_{ID, Name, Salary} (INSTRUCTOR)$

Output:

ID	Name	Salary
10101	John	65000
12121	Robin	90000
25252	Alya	40000
26589	Yusuf	95000
54789	Ravi	60000
78787	Raj	87000
87458	Jayant	75000
76985	Pratik	89000
12547	Neil	80000

2. Project (Π)

Example 2: Find the name of all instructors in the Computer Science department.

Solution:

$\Pi_{\text{Name}} (\sigma_{\text{Dept_Name} = \text{"Computer Science"}} (\text{INSTRUCTOR}))$

INSTRUCTOR			
ID	Name	Dept_Name	Salary
10101	John	Biology	65000
12121	Robin	Computer Science	90000
25252	Alya	Electrical	40000
26589	Yusuf	Finance	95000
54789	Ravi	Music	60000
78787	Raj	Physics	87000
87458	Jayant	History	75000
76985	Pratik	Computer Science	89000
12547	Neil	Finance	80000

Output:

Name

Robin

Pratik

3. Union (U)

- ☆ Any relation is a **set**.
- ☆ Similar to **union** operation in Set Theory.
- ☆ It is a **binary** operator.
- ☆ It is a set of all objects that are a member of A, or B, or both.
- ☆ Like project, the **duplicate rows** are eliminated.
- ☆ It is denoted by U.
- ☆ Syntax: $\Pi_{\text{Column}}(\text{Relation_1}) \cup \Pi_{\text{Column}}(\text{Relation_2})$



3. Union (U)

R	Alphabets
	A
	B
	C
	E
	F

S	Characters
	A
	B
	\$
	E
	F
	!
	#

R U S
A
B
C
E
F
\$
!
#

3. Union (U)

Example 1: List all customer names associated with the bank either as an account holder or a loan borrower.

Solution: $\Pi_{Cu_Name}(Depositor) \cup \Pi_{Cu_Name}(Borrower)$

DEPOSITOR	Cu_Name	Acc_No
R	Tom	A-101
	Amy	A-502
	Rose	A-304
	John	A-689

BORROWER	Cu_Name	Loan_No
S	John	L-201
	Smith	L-658
	Rose	L-254
	Jack	L-547

Cu_Name
Tom
Amy
Rose
John
Smith
Jack

Example – University Database

Instructor (ID, Name, Dept_Name, Salary)

Course (Course_ID, Title, Dept_Name, Credits)

Department (Dept_Name, Building, Budget)

Section (Course_ID, Sec_ID, Semester, Year, Building, Room_No, Time_slot_ID)

Teaches (ID, Course_ID, Sec_ID, Semester, Year)

Student (ID, Name, Dept_Name, Tot_Cred)

Advisor (S_ID, I_ID)

Takes (ID, Course_ID, Sec_ID, Semester, Year, Grade)

Classroom (Building, Room_Number, Capacity)

Time_Slot (Time_Slot_ID, Day, Start_Time, End_Time)



3. Union (U)

Example 2: Find the set of all courses taught in the Fall 2009 semester, the Spring 2010 semester, or both.

Solution:

To find the set of all courses taught in the Fall 2009 semester, we write:

$$\Pi_{\text{Course_id}} (\sigma_{\text{Semester} = \text{"Fall"} \wedge \text{year} = 2009} (\text{SECTION}))$$

To find the set of all courses taught in the Spring 2010 semester, we write:

$$\Pi_{\text{Course_id}} (\sigma_{\text{Semester} = \text{"Spring"} \wedge \text{year} = 2010} (\text{SECTION}))$$

To answer the query, we need the union of these two sets:

$$\Pi_{\text{Course_id}} (\sigma_{\text{Semester} = \text{"Fall"} \wedge \text{year} = 2009} (\text{SECTION})) \cup \Pi_{\text{Course_id}} (\sigma_{\text{Semester} = \text{"Spring"} \wedge \text{year} = 2010} (\text{SECTION}))$$

Student $\cap$ project

name	rollno
Sai	12

Student

name	rollno
Sai	12
Ramesh	13
Suresh	14

Project

name	rollno
Sai	12
Shiva	30
megendra	40

Student $\cup$ project

name	rollno
Sai	12
Ramesh	13
Suresh	14
Shiva	30
megendra	40

Syntax: table1 - table2

Student - project
Project - Student

name	rollno
Ramesh	13
Suresh	14

4. Set Difference (-)

Example 1: List all customer names those who have a deposit account but not availed loan.

DEPOSITOR	Cu_Name	Acc_No	BORROWER	Cu_Name	Loan_No
R	Tom	A-101	S	John	L-201
	Amy	A-502		Smith	L-658
	Rose	A-304		Rose	L-254
	John	A-689		Jack	L-547

$R - S$

Cu_Name
Tom
Amy

Solution: $\Pi_{Cu_Name}(\text{Depositor}) - \Pi_{Cu_Name}(\text{Borrower})$

4. Set Difference (-)

Example 2: Find all the courses taught in the Fall 2009 semester but not in Spring 2010.

Solution:

To find the set of all courses taught in the Fall 2009 semester, we write:

$$\Pi_{\text{Course_id}} (\sigma_{\text{Semester} = \text{"Fall"} \wedge \text{year} = 2009} (\text{SECTION}))$$

To find the set of all courses taught in the Spring 2010 semester, we write:

$$\Pi_{\text{Course_id}} (\sigma_{\text{Semester} = \text{"Spring"} \wedge \text{year} = 2010} (\text{SECTION}))$$

To answer the query, we need the minus of these two sets:

$$\Pi_{\text{Course_id}} (\sigma_{\text{Semester} = \text{"Fall"} \wedge \text{year} = 2009} (\text{SECTION})) - \Pi_{\text{Course_id}} (\sigma_{\text{Semester} = \text{"Spring"} \wedge \text{year} = 2010} (\text{SECTION}))$$

5. Cartesian Product (X)

- ☆ Cartesian product associates **every** tuple of R_1 with **every** tuple of R_2 .
 - ☆ It is a **binary** operation.
 - ☆ It is denoted by cross (X) symbol.
 - ☆ $R_1 \times R_2$ = All possible pairing.
 - ☆ Same attribute may appear in R_1 and R_2 .
 - ☆ $R = \text{Depositor} \times \text{Borrower}$.
-

5. Cartesian Product (X)

Example: Perform Depositor X Borrower.

DEPOSITOR	
Cu_Name	Acc_No
Tom	A-101
Rose	A-304

BORROWER	
Cu_Name	Loan_No
John	L-201
Smith	L-658
Rose	L-254
Jack	L-547

DEPOSITOR X BORROWER			
Cu_Name	Acc_No	Cu_Name	Loan_No
Tom	A-101	John	L-201
Tom	A-101	Smith	L-658
Tom	A-101	Rose	L-254
Tom	A-101	Jack	L-547
Rose	A-304	John	L-201
Rose	A-304	Smith	L-658
Rose	A-304	Rose	L-254
Rose	A-304	Jack	L-547

5. Cartesian Product (X)

$\sigma_{\text{Dept_Name} = \text{"Physics"}} (\text{INSTRUCTOR X TEACHES})$

INSTRUCTOR.ID	Name	Dept_Name	Salary	TEACHES.ID	Course_ID	Sec_ID	Semester	Year
10101	John	Biology	65000	10101	BIO-108	1	Summer	2009
10101	John	Biology	65000	20202	CS-103	1	Spring	2010
10101	John	Biology	65000	78787	PHY-101	2	Fall	2011
10101	John	Biology	65000	12345	EE-203	1	Spring	2009
78787	Raj	Physics	87000	10101	BIO-108	1	Summer	2009
78787	Raj	Physics	87000	20202	CS-103	1	Spring	2010
78787	Raj	Physics	87000	78787	PHY-101	2	Fall	2011
78787	Raj	Physics	87000	12345	EE-203	1	Spring	2009

5. Cartesian Product (X)

Example : Find the names of all instructors in the Physics department together with the course id of all courses they taught.

Solution:

$\sigma_{\text{Dept_Name} = \text{"Physics"}} (\text{INSTRUCTOR} \times \text{TEACHES})$

$\sigma_{\text{Instructor.ID} = \text{Teaches.ID}} (\sigma_{\text{Dept_Name} = \text{"Physics"}} (\text{INSTRUCTOR} \times \text{TEACHES}))$

$\Pi_{\text{name, course_id}} (\sigma_{\text{Instructor.ID} = \text{Teaches.ID}} (\sigma_{\text{Dept_Name} = \text{"Physics"}} (\text{INSTRUCTOR} \times \text{TEACHES})))$

[or]

$\Pi_{\text{name, course_id}} (\sigma_{\text{Instructor.ID} = \text{Teaches.ID}} ((\sigma_{\text{Dept_Name} = \text{"Physics"}} (\text{INSTRUCTOR})) \times \text{TEACHES}))$

6. Rename (ρ)

- ☆ Relations in the database have names.
 - ☆ The results of relational-algebra expressions **do not have a name**.
 - ☆ It is useful to be able to give them names.
 - ☆ It is a unary operation.
 - ☆ It is denoted by the lowercase Greek letter **rho** (ρ).
 - ☆ Syntax: $\rho_x(E)$

-

Set Intersection ($\cap$)

- ☆ Any relation is a **set**.
 - ☆ Similar to **intersection** operation in Set Theory.
 - ☆ It is a **binary** operator.
 - ☆ It is a set of all objects that are a member of both A and B.
 - ☆ It is denoted by $\cap$.
 - ☆ Syntax: $\Pi_{\text{Column}}(\text{Relation}_1) \cap \Pi_{\text{Column}}(\text{Relation}_2)$
-

Set Intersection ($\cap$)

Example 1: Find the names of all customers who have deposited money and also availed loan.

Solution: $\Pi_{\text{Cu_Name}}(\text{Depositor}) \cap \Pi_{\text{Cu_Name}}(\text{Borrower})$

DEPOSITOR	Cu_Name	Acc_No
R	Tom	A-101
	Amy	A-502
	Rose	A-304
	John	A-689

BORROWER	Cu_Name	Loan_No
S	John	L-201
	Smith	L-658
	Rose	L-254
	Jack	L-547

$R \cap S$

Cu_Name
Rose
John

Set Intersection ($\cap$)

Example 2: Find the set of all courses taught in the Fall 2009 semester and the Spring 2010 semesters.

Solution:

To find the set of all courses taught in the Fall 2009 semester, we write:

$\Pi_{\text{Course_id}} (\sigma_{\text{Semester} = \text{"Fall"} \wedge \text{year} = 2009} (\text{SECTION}))$

To find the set of all courses taught in the Spring 2010 semester, we write:

$\Pi_{\text{Course_id}} (\sigma_{\text{Semester} = \text{"Spring"} \wedge \text{year} = 2010} (\text{SECTION}))$

To answer the query, we need the intersection of these two sets:

Join Operation

- ☆ Join Operation = Cartesian Product + Selection Process.
- ☆ Join Operation pairs two tuples from different relations, if and only if the join condition is satisfied.

Inner Join

- ☆ Only those tuples that satisfy the matching criteria are included.
- ☆ This discards row(s) of either table that do not match any row of the other table.
- ☆ The rest are excluded.
- ☆ Types of Inner Join:
 1. Theta Join ($\bowtie_{\theta}$)
 2. Equi Join ($\bowtie_{\theta}$)
 3. Natural Join ($\bowtie_*$)

1. Theta Join ($\bowtie_{\theta}$)

- ☆ A **general case** of join operation.
- ☆ Also known as **conditional join**.
- ☆ Denoted by theta symbol ($\bowtie_{\theta}$).
- ☆ Used to join two or more relations on **some conditions**.
- ☆ Syntax: $P \bowtie_{\theta} Q$.
- ☆ θ is the **predicate**.
- ☆ Conditional operators: $=$, $\neq$, $<$, $\leq$, $>$, and $\geq$.

1. Theta Join ($\bowtie_{\theta}$)

Example: Perform $S_1 \bowtie_{S_1.S_ID > R_1.S_ID} R_1$

S1	S_ID	S_NAME	RATING	AGE
	22	Tom	7	20
	31	Amy	8	19
	58	Rose	10	21

R1	S_ID	B_ID	DATE
	22	101	10-May-21
	58	103	05-Jan-22

Solution:

$S_1 \bowtie_{S_1.S_ID > R_1.S_ID} R_1$						
S_ID	S_NAME	RATING	AGE	S_ID	B_ID	DATE
31	Amy	8	19	22	101	10-May-21
58	Rose	10	21	22	101	10-May-21

2. Equi Join ($\bowtie_{\theta}$)

Example: Perform $S_1 \bowtie_{S_1.S_ID = R_1.S_ID} R_1$

S1	S_ID	S_NAME	RATING	AGE
	22	Tom	7	20
	31	Amy	8	19
	58	Rose	10	21

R1	S_ID	B_ID	DATE
	22	101	10-May-21
	58	103	05-Jan-22

Solution:

$S_1 \bowtie_{S_1.S_ID = R_1.S_ID} R_1$						
S_ID	S_NAME	RATING	AGE	S_ID	B_ID	DATE
22	Tom	7	20	22	101	10-May-21
58	Rose	10	21	58	103	05-Jan-22

3. Natural Join ($\bowtie_*$)

- ☆ A **special** case of Equi Join.
- ☆ Can be performed only if there is a **common attribute**.
- ☆ The **name and domain** of the attributes must be the **same**.
- ☆ Does not use any **comparison operator**.
- ☆ **Natural Join = Cartesian Product + Selection + Projection.**
- ☆ Does not do **concatenation** like **Cartesian Product**.
- ☆ Denoted by $\bowtie_*$.
- ☆ Example: $P \bowtie_* Q$

3. Natural Join ($\bowtie_*$)

Example: Perform $S_1 \bowtie_* R_1$

S1	S_ID	S_NAME	RATING	AGE
	22	Tom	7	20
	31	Amy	8	19
	58	Rose	10	21

R1	S_ID	B_ID	RATING	DATE
	22	101	8	10-May-21
	58	103	10	05-Jan-22

Solution:

$S_1 \bowtie_* R_1$					
S_ID	S_NAME	RATING	AGE	B_ID	DATE
58	Rose	10	21	103	05-Jan-22

Extended Relational Algebra Functions

The generalized projection operation extends the projection operation by allowing arithmetic functions to be used in the projection list. The generalized projection has the form,

$$\Pi_{F_1, F_2, \dots, F_n}(E)$$

where,

E : Relation algebra expression

$F_1, F_2, \dots, F_n$: An arithmetic expression involving constants and attributes of E .

Extended Relational Algebra Functions

Example : *Display name and percentage of students.*

$\Rightarrow \Pi_{\text{Name, Total/5}}$ as **Percentage (Student)**

Student		
Roll_no	Name	Total
1	Hari	350
2	Om	400
3	Smith	375
4	Jay	425

Name	Percentage
Hari	70
Om	80
Smith	75
Jay	85

Extended Relational Algebra

Aggregate Functions

Functions

Employee_salary			
Emp_code	Name	Salary	Department
E0001	Hari	10000	Computer
E0002	Om	7000	IT
E0003	Smith	8000	Computer
E0004	Jay	5000	IT

➤ Examples :

1) Calculate total salary paid to employees.

$\Rightarrow G_{\text{sum}(\text{Salary})}(\text{Employee_salary})$

The symbol ' G ' is the letter G in calligraphic font; read it as 'calligraphic G'.

The G signifies that aggregate function is to be applied, and its subscript specifies the aggregate operation to be applied. The result of above query is shown in Fig. 3.23.

Salary
30000

Extended Relational Algebra Functions

2) *Display average salary of employees.*

$\Rightarrow G_{avg(Salary)}(Employee_salary)$

The result of above query is shown in Fig. 3.24.

Salary
7500

3) *Count number of distinct departments in Employee_salary relation.*

$\Rightarrow G_{count-distinct(Department)}(Employee_salary)$

The above query return the value 2.

Extended Relational Algebra Functions

4) *Display total salary of employees department wise.*

⇒ department $g_{\text{sum}(\text{Salary})}(\text{Employee_salary})$

The result of above query is shown in Fig. 3.25.

Department	Salary
Computer	18000
I.T.	12000

Extended Relational Algebra Functions

Outer-Join

The outer-join operation is an extension of the join operation to deal with missing information.

Employee	
Emp_name	City
Hari	Pune
Om	Mumbai
Smith	Nashik
Jay	Solapur

Employee_salary		
Emp_name	Department	Salary
Hari	Computer	10000
Om	IT	7000
Bill	Computer	8000
Jay	IT	5000

Employee $\bowtie$ Employee_salary

3755 A 1

Extended Relational Algebra Functions

Emp_name	City	Department	Salary
Hari	Pune	Computer	10000
Om	Mumbai	IT	7000
Jay	Solapur	IT	5000

To avoid this loss of information, we can use outer join operation.

Types of outer-join

- a) Left outer-join.
- b) Right outer-join.
- c) Full outer-join.

Extended Relational Algebra Functions

a) Left outer-join :

It is denoted by $\bowtie$. It takes all tuples in the left relation that did not match with any tuple in the right relation, pads the tuples with null values for all other attributes from the right relation and adds them to the result of the natural-join.

The result of the expression -

Employee $\bowtie$ Employee_salary is

Emp_name	City	Department	Salary
Hari	Pune	Computer	10000
Om	Mumbai	IT	7000
Jay	Solapur	IT	5000
Smith	Nashik	null	null

Employee	
Emp_name	City
Hari	Pune
Om	Mumbai
Smith	Nashik
Jay	Solapur

Employee_salary		
Emp_name	Department	Salary
Hari	Computer	10000
Om	IT	7000
Bill	Computer	8000
Jay	IT	5000

'Left Outer Join'

it gives the matching rows and the rows which are in left table but not in right table.

Emp.no	E_name	d_name	Loc
E ₁	Varun	IT	Delhi
E ₂	Amsrit	HR	Hyd
E ₃	Ravi	IT	Delhi
E ₄	Nitin	—	—

Emp			Dept		
Emp_no	E_name	Dept.no	Dept-No	D_name	Loc
E ₁	Varun	D ₁	→ D ₁	IT	Delhi
E ₂	Amsrit	D ₂	→ D ₂	HR	Hyd
E ₃	Ravi	D ₁	→ D ₃	Finance	Pune
E ₄	Nitin	—			

Select emp.no, e.name, d.name, Loc from emp left outer join dept
On (emp.dept.no = dept.dept.no)

Extended Relational Algebra Functions

b) Right outer-join

The right outer-join is denoted by $\bowtie\!\!\!\lrcorner$. It pads tuples from the right relation that did not match with any tuple from the left relation with nulls and adds them to the result of the natural-join.

The result of the expression

$\text{Employee} \bowtie\!\!\!\lrcorner \text{Employee_salary}$

Emp_name	City	Department	Salary
Hari	Pune	Computer	10000
Om	Mumbai	IT	7000
Jay	Solapur	IT	5000
Bill	null	Computer	8000

Employee	
Emp_name	City
Hari	Pune
Om	Mumbai
Smith	Nashik
Jay	Solapur

Employee_salary		
Emp_name	Department	Salary
Hari	Computer	10000
Om	IT	7000
Bill	Computer	8000
Jay	IT	5000

Extended Relational Algebra Functions

c) Full outer-join :

It is denoted by $\bowtie$. It pads tuples from the left relation that did not match with any tuple from the right relation, as well as tuples from the right relation that did not match with any tuple from the left relation and adds them to the resultant relation.

The result of the expression

`Employee  $\bowtie$  Employee_salary .`

Emp_name	City	Department	Salary
Hari	Pune	Computer	10000
Om	Mumbai	IT	7000
Jay	Solapur	IT	5000
Smith	Nashik	null	null
Bill	null	Computer	8000

Types of SQL Commands

DDL data definition language

Create table , Create view, Create Index

Alter table

Drop table, view, Index

SQL: Data Definition Language (DDL)

- ❑ **Data Definition Language (DDL)** is used for creating and modifying the database objects such as tables, indices, views and users.
- ❑ **DDL Commands** are used to define the structure and schema of the database
- ❑ All the command of DDL are *auto-committed* that means it permanently save all the changes in the database.
- ❑ **Data Definition Language (DDL) Commands:**
 - ❑ **CREATE** - to create new table or database
 - ❑ **ALTER** - to alter (or modify) the structure of table
 - ❑ **DROP** - to delete a table from database
 - ❑ **TRUNCATE** - to delete all records from table
 - ❑ **RENAME** - to rename a table



CREATE DATABASE statement is used to create a database in RDBMS.

Syntax:

CREATE DATABASE database_name;

Example:

CREATE DATABASE my_db; ✓

Note:

❖ **SHOW statement:** To see existing **databases** and **tables**

SHOW databases;

SHOW tables;

❖ **USE statement:** To use or select any existing database

Syntax **USE database_name;**

Example: **USE my_db;**

➤ **CREATE TABLE** statement is used **to create a new table in a database.**

- ▣ It specifies **column names** of the table, its **data types** (e.g. varchar, integer, date, etc.) and can also specify **integrity constraints** (e.g. Primary key, foreign key, not null, unique).

Syntax:

```
CREATE TABLE table_name  
(  
    column_name1  datatype,  
    column_name2  datatype,  
    ...           ...  
    column_name n  datatype,  
    (Integrity Constraints)  
);
```



Example 1:

```
CREATE TABLE Emp(  
    Emp_ID int,  
    Name varchar(20),  
    Age int,  
    Address varchar(100),  
    Salary numeric(10,2)  
);
```



Emp_ID	Name	Age	Address	Salary

Example 2:

```
CREATE TABLE branch(  
    ✓ branch-name varchar(15),  
    branch-city varchar(30),  
    assets integer,  
    primary key (branch-name),  
    check(assets>0)  
);
```



<u>branch-name</u>	<u>branch-city</u>	<u>assets</u>

Note:



DESC (or DESCRIBE) statement: To describe the details of the table structure

Syntax: **DESC** *table_name*;

Example: **DESC** *Emp*;

SQL DATA TYPES



- ❑ **Each column** in a database **table** is required to have a **name** and a **data type**.
- ❑ The **data type** is a guideline for SQL to understand what type of data is expected inside of each column, and it also identifies how SQL will interact with the stored data.
- ❑ ✓ **Note:** Data types might have different names in different database. And even if the name is the same, the size and other details may be different! Always check the documentation!



Data type	Description
CHAR(size)	A FIXED length string (can contain letters, numbers, and special characters), with user-specified length size.
VARCHAR(size)	A VARIABLE length string (can contain letters, numbers, and special characters), with user-specified length size. Full Form Character Varying
INT / INTEGER	A medium Integer (Signed range is from -2,147,483,648 to 2,147,483,647. Unsigned range is from 0 to 4294967295). 4 bytes
SMALLINT	A small integer (Signed range is from -32,768 to 32,767. Unsigned range is from 0 to 65,535.) 2 bytes
BIGINT	A large integer (Signed range is from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,808. Unsigned range is from 0 to 18446744073709551615). 8 bytes
NUMERIC(size,d)/ DECIMAL(size,d)/ FLOAT(size,d)	A small floating point number. Its size parameter specifies the total number of digits. The number of digits after the decimal point is specified by <i>d</i> parameter. For NUMRIC(3,1), 46.2 can be stored but not 466.0 or 0.32
REAL, DOUBLE PRECISION	Floating point and double-precision floating point numbers, with machine-dependent precision.
FLOAT(n)	A floating point number, with user-specified precision of at least <i>n</i> digits.
DOUBLE(size, d)	A large floating point number. The total number of digits is specified in size. The number of digits after the decimal point is specified in the <i>d</i> parameter





Data type	Description
DATE ✓	A date. Format: <u>YYYY-MM-DD</u> . The supported range is from '1000-01-01' to '9999-12-31'
TIME	A time. Format: hh:mm:ss. The supported range is from '-838:59:59' to '838:59:59'
DATETIME / TIMESTAMP	A date and time combination. Format: YYYY-MM-DD hh:mm:ss.
YEAR	A year in four-digit format. Values allowed in four-digit format: 1901 to 2155, and 0000.

Note: There are many other data types.....

SQL Constraints

- ❑ **SQL constraints** are used to specify rules for the data in a table.
- ❑ **Constraints** are used to limit the type of data that can go into a table. This ensures the accuracy and reliability of the data in the table. If there is any violation between the constraint and the data action, the action is aborted.
- ❑ **Constraints** can be **column level** or **table level**.
 - ❑ **Column level** constraints apply to a column
 - ❑ **Table level** constraints apply to the whole table.
- ❑ **Constraints** can be specified:
 - ❑ when a table is created with the **CREATE TABLE** statement, or
 - ❑ we can use the **ALTER TABLE** statement to create constraints even after the table is created.

SQL Constraints:

- ✓ **NOT NULL**: Ensures that a column cannot have NULL value.
- ▣ **DEFAULT**: Provides a default value for a column when none is specified.
- ▣ **UNIQUE**: Ensures that all values in a column are different.
- ▣ **PRIMARY KEY**: A combination of a NOT NULL and UNIQUE. Uniquely identifies each row in a table
- ▣ **FOREIGN KEY**: Uniquely identifies a row/record in another table.
- ▣ **CHECK**: Ensures that all the values in a column satisfies certain conditions
- ▣ **INDEX**: Used to create and retrieve data from the database very quickly

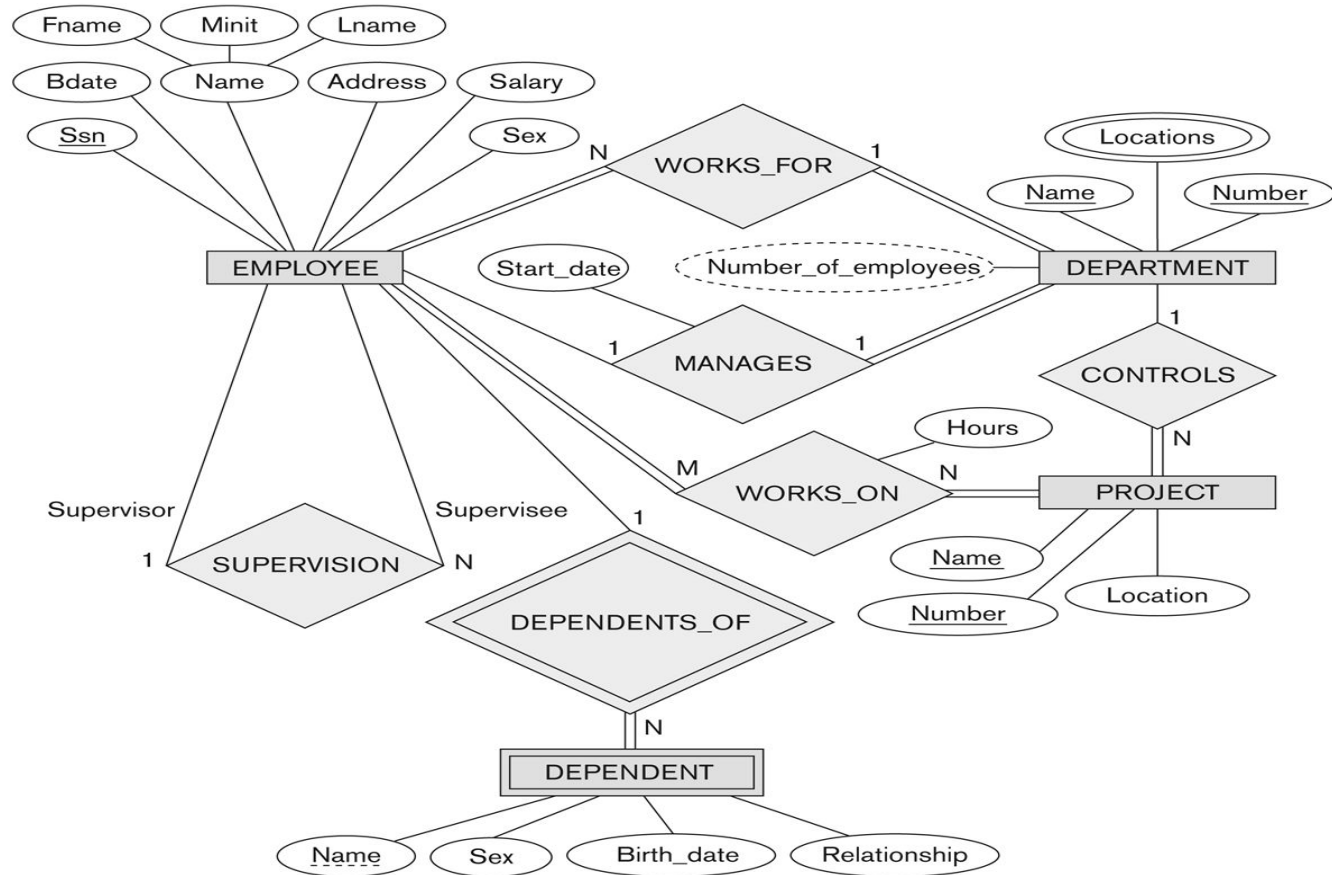


Figure 3.2

An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

CREATE TABLE EMPLOYEE

(Fname	VARCHAR(15)	NOT NULL,
Minit	CHAR,	
Lname	VARCHAR(15)	NOT NULL,
Ssn	CHAR(9)	NOT NULL,
Bdate	DATE,	
Address	VARCHAR(30),	
Sex	CHAR,	
Salary	DECIMAL(10,2),	
Super_ssn	CHAR(9),	
Dno	INT	NOT NULL,

PRIMARY KEY (Ssn),**CREATE TABLE DEPARTMENT**

(Dname	VARCHAR(15)	NOT NULL,
Dnumber	INT	NOT NULL,
Mgr_ssn	CHAR(9)	NOT NULL,
Mgr_start_date	DATE,	

PRIMARY KEY (Dnumber),**UNIQUE (Dname),****FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn));****Figure 6.1**

SQL CREATE
TABLE data
definition statements
for defining the
COMPANY schema
from Figure 5.7.

```

        FOREIGN KEY (mgr_ssn) REFERENCES EMPLOYEE(SSN) );
CREATE TABLE DEPT_LOCATIONS
    ( Dnumber                INT                NOT NULL,
      Dlocation              VARCHAR(15)        NOT NULL,
      PRIMARY KEY (Dnumber, Dlocation),
      FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber) );
CREATE TABLE PROJECT
    ( Pname                  VARCHAR(15)        NOT NULL,
      Pnumber                INT                NOT NULL,
      Plocation              VARCHAR(15),
      Dnum                   INT                NOT NULL,
      PRIMARY KEY (Pnumber),
      UNIQUE (Pname),
      FOREIGN KEY (Dnum) REFERENCES DEPARTMENT(Dnumber) );
CREATE TABLE WORKS_ON
    ( Essn                   CHAR(9)            NOT NULL,
      Pno                    INT                NOT NULL,
      Hours                  DECIMAL(3,1)       NOT NULL,
      PRIMARY KEY (Essn, Pno),
      FOREIGN KEY (Essn) REFERENCES EMPLOYEE(SSN),
      FOREIGN KEY (Pno) REFERENCES PROJECT(Pnumber) );

```

Example:

CREATE TABLE Emp

(

Emp_ID **int** **NOT NULL**,

Name **varchar(20)** **NOT NULL**,

Age **int** **NOT NULL**,

Address **varchar(100)** **DEFAULT** 'India',

Salary **numeric(10,2)** **NOT NULL**,

DeptID **int**,

PRIMARY KEY(Emp_ID),

FORIEGN KEY (Dept_ID) **REFERENCES** Dept(Dept_ID),

CHECK(Age >= 18)

);

2. ALTER

- ❑ ALTER TABLE statement is **used to modify the structure of the existing table**
- ❑ It is used to add, delete, modify or rename columns in an existing table.
- ❑ It is also used to add and drop various constraints on an existing table.

1. ALTER TABLE – ADD COLUMN

- For adding new columns in a table

2. ALTER TABLE - DROP COLUMN

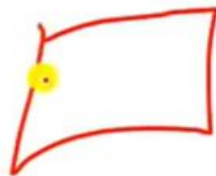
- For removing existing columns in table

3. ALTER TABLE - MODIFY COLUMN

- To modify existing columns in a table

4. ALTER TABLE – RENAME COLUMN

- To rename an existing column in a table



1. ALTER TABLE – ADD COLUMN



- For adding new columns in a table ✓

- Syntax:

```
ALTER TABLE table_name  
ADD column_name(s) datatype(s);
```

Example:

```
ALTER TABLE Emp  
ADD Email varchar(100);
```



Emp_ID	Name	Age	Address	Salary	Email



2. ALTER TABLE - DROP COLUMN

- For removing existing columns in a table
- Syntax:

```
ALTER TABLE table_name  
DROP column_name(s);
```

Example:

```
ALTER TABLE Emp  
DROP Email;
```

Emp_ID	Name	Age	Address	Salary

3. ALTER TABLE - **MODIFY** COLUMN

- To modify existing columns in a table
 - To change data type of any column or to modify its size.
- Syntax:

ALTER TABLE *table_name* .
MODIFY *column_name* *datatype*;

Example:

```
ALTER TABLE Emp  
MODIFY Name varchar(100);
```

Check it
with **DESC**
Command

4. ALTER TABLE – **RENAME** COLUMN



- To rename an existing column in a table

- Syntax:

ALTER TABLE *table_name*

RENAME old_column_name **To** new_column_name;

Example:

ALTER TABLE *Emp*

MODIFY *Address* **To** *Location*;



Emp_ID	Name	Age	Location	Salary

3. DROP

➤ **DROP TABLE** statement completely removes a table from the database.

➤ This command will destroy the table structure and the data stored in it.

Syntax:

DROP TABLE table_name;

Example:

DROP TABLE Emp;



DROP command is also be used on Databases.

DROP DATABASE statement is used to delete the complete database.

Syntax:

DROP DATABASE database_name;

Example:

DROP DATABASE my_db;

4. TRUNCATE



- ❑ **TRUNCATE TABLE** statement is used to remove all rows (complete data) from a **table**. It is similar to the DELETE statement with no WHERE clause.
- ❑ **TRUNCATE TABLE Vs DROP TABLE**
 - ❑ DROP TABLE command can also be used to delete complete table but it deletes table structure too. TRUNCATE TABLE doesn't delete the structure of the table.
- ❑ **Syntax:**

TRUNCATE TABLE *table_name*;

Emp

Emp_ID	Name	Age	Location	Salary
1	Rahul	32	Delhi	2000
2	Kamal	25	Pune	1500

Example:

TRUNCATE TABLE *Emp*;



Emp_ID	Name	Age	Location	Salary



5. RENAME

- RENAME TABLE statement is used to change the name of a table.

- Syntax:

RENAME TABLE old_table _name **To** new_table_name;

Example:

RENAME TABLE *Emp* **To** *Employee*;

Queries (SELECT)

- Retrieval of some information from the database
- **Result Set** contains the “answer” for the query.
- Result “Sets” can contain duplicate rows
- Single value Result Sets are considered to be one row with one column
- Result Sets can be empty

Simple Format

```
SELECT <columns>  
FROM    <tables>  
WHERE   <condition>
```

Where:

- **Columns** = list of attributes to be retrieved
- **Tables** = list of relations needed to process the query
- **Condition** = a boolean expression that identifies which tuples are to be retrieved

Example

```
SELECT <columns>  
FROM    <tables>  
WHERE   <condition>
```

Example:

```
SELECT title, description  
FROM    course  
WHERE   dcode='COMP'
```

Important Note #2

- If you are trying to select on a field where the data type is VARCHAR(), then this select will do:
 - `SELECT * FROM foo WHERE name='BOB'`
- If you are trying to select on a field where the data type is CHAR(5), then this select will do:
 - `SELECT * FROM foo WHERE name='BOB '`

Important Note #3

- Remember that selects on CHAR and VARCHAR types are case sensitive. The SELECTS:
 - `SELECT * FROM foo WHERE name='BOB'`
 - `SELECT * FROM foo WHERE name='Bob'`
 - `SELECT * FROM foo WHERE name='BoB'`
- are all different to the DBMS.

Our database (simple version)

Student (number, street, city, province, postal_code, name, date_of_birth)

Faculty (number, name, rank, phone, office, email, dcode, salary)

Department (code, name, start_date, end_date, fnum)

Section (number, cnum, dcode, term, slot, faculty_num)

Course (number, dcode, title, description)

Enrolled (student_num, section_num, cnum, dcode)

Dept_phones (dcode, phone_number)

Example 1

List course number, department code, title and description of each course.

```
SELECT number, dcode, title, description  
FROM course
```

OR

```
SELECT * ← "select all attributes"  
FROM course
```

Example 2

Find the faculty number and name of each faculty member in Computer Science with the rank of “Assistant”

```
SELECT number, name  
FROM faculty  
WHERE dcode = 'COMP' and rank = 'Assistant'
```

Example #3

Which computer science courses are being taught in the second term?

SELECT cnum

FROM section

WHERE dcode = 'COMP' and term = 'X2'

The result has only one column called “cnum”

Views

```
CREATE VIEW view name as  
SELECT column name, cloumn name  
FROM table name  
WHERE column name = expression list ;
```

Views

Create view on Book table which contains two fields title, and author name.

```
SQL> create view V_Book as  
      select Title, Author_name  
      from Book;
```

View created.

```
SQL> select * from V_Book;
```

Title	Author_name
Oracle	Arora
DBMS	Basu
DOS	Sinha
ADBMS	Basu
Unix	Kapoor

Set Operators

- ✓ Set operators allow you to combine the results of two or more SELECT (queries) statements into a single result set based on set theory principles.
- ✓ These operators allow you to perform operations such as combining, intersecting, or subtracting sets of data.

Set Operators



UNION



UNION
ALL



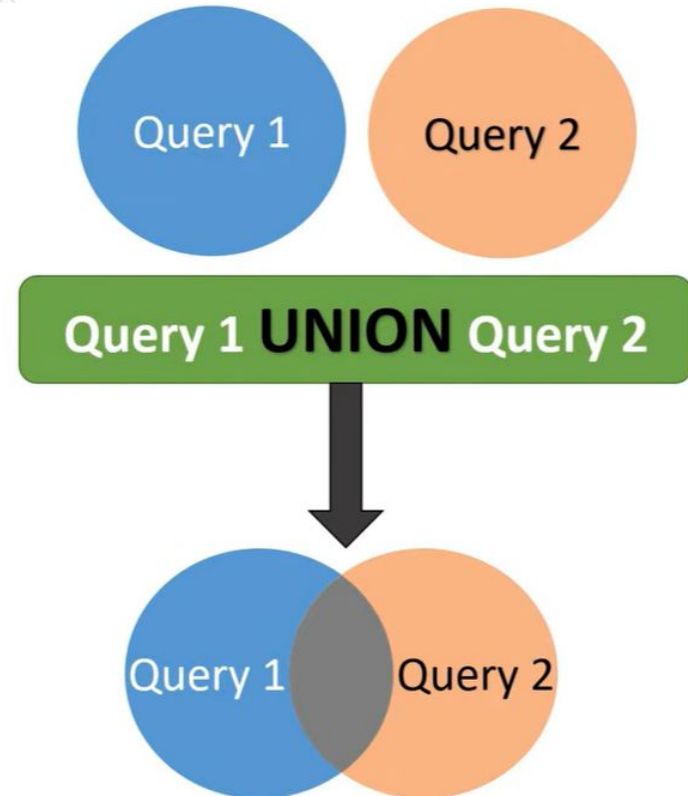
INTERSECT



MINUS
Or
EXCEPT

UNION

- The UNION operator combines the result sets of two or more SELECT statements into a single result set.
- It removes duplicate rows from the result set.



UNION

SYNTAX :

```
SELECT column_name(s) FROM table1  
UNION  
SELECT column_name(s) FROM table2  
UNION  
SELECT column_name(s) FROM table3;  
.....
```

Rules to use SET Operators

- Every SELECT statement must have the same number of columns
- The columns must also have similar data types.
- The columns in every SELECT statement must also be in the same order.

DataTypes			
int(10)	varchar(15)	Int(5)	varchar(15)
Id	Name	Age	Phone

Table for Query 1

DataTypes			
int(10)	varchar(15)	Int(5)	varchar(15)
Id	Name	Age	Phone

Table for Query 2

SQL Questions



Question-1

- Write a SQL query to return all the details of the employees from the employee and manager table. Also remove duplicate records.

```

1 SELECT * FROM employee
2 UNION
3 SELECT * FROM manager;

```

id	name	salary	department	dob
101	Jack	2000	HR	1997-05-19
102	Jack		HR	
103	Mack	6000	Developer	1997-03-10
104	Peter	4000	Tester	1998-07-16
105	Tom	3000	HR	1998-11-03
106	Leo	2500	Developer	1997-10-10
107	Roger	5300	Accounts	1997-06-17
108	Mike	2000		1998-03-09
109	Paul	4800	Developer	1998-12-28
110	Hannah	2000	Accounts	1999-09-26
101	Nancy	4000	HR	1999-12-30
102	Lucy	6500	HR	1995-09-09
104	Charlie	9500	Tester	1998-06-24

employee

id	name	salary	department	dob
101	Jack	2000	HR	1997-05-19
102	Jack		HR	
103	Mack	6000	Developer	1997-03-10
104	Peter	4000	Tester	1998-07-16
105	Tom	3000	HR	1998-11-03
106	Leo	2500	Developer	1997-10-10
107	Roger	5300	Accounts	1997-06-17
108	Mike	2000		1998-03-09
109	Paul	4800	Developer	1998-12-28
110	Hannah	2000	Accounts	1999-09-26

manager

id	name	salary	department	dob
101	Nancy	4000	HR	1999-12-30
102	Lucy	6500	HR	1995-09-09
103	Mack	6000	Developer	1997-03-10
104	Charlie	9500	Tester	1998-06-24
105	Tom	3000	HR	1998-11-03

Query 1

Query 2

Query 1 **UNION ALL** Query 2



Query 1



Query 2

UNION ALL

SYNTAX :

```
SELECT column_name(s) FROM table1
```

```
UNION ALL
```

```
SELECT column_name(s) FROM table2
```

```
UNION ALL
```

```
.....;
```

```
1 SELECT * FROM employee
2 UNION ALL
3 SELECT * FROM manager;
```

result Grid Filter Rows: Export: Wrap Cell Contents:

id	name	salary	department	dob
101	Jack	2000	HR	1997-05-19
102	Jack		HR	
103	Mack	6000	Developer	1997-03-10
104	Peter	4000	Tester	1998-07-16
105	Tom	3000	HR	1998-11-03
106	Leo	2500	Developer	1997-10-10
107	Roger	5300	Accounts	1997-06-17
108	Mike	2000		1998-03-09
109	Paul	4800	Developer	1998-12-28
110	Hannah	2000	Accounts	1999-09-26
101	Nancy	4000	HR	1999-12-30
102	Lucy	6500	HR	1995-09-09
103	Mack	6000	Developer	1997-03-10
104	Charlie	9500	Tester	1998-06-24
105	Tom	3000	HR	1998-11-03

employee

id	name	salary	department	dob
101	Jack	2000	HR	1997-05-19
102	Jack		HR	
103	Mack	6000	Developer	1997-03-10
104	Peter	4000	Tester	1998-07-16
105	Tom	3000	HR	1998-11-03
106	Leo	2500	Developer	1997-10-10
107	Roger	5300	Accounts	1997-06-17
108	Mike	2000		1998-03-09
109	Paul	4800	Developer	1998-12-28
110	Hannah	2000	Accounts	1999-09-26

manager

id	name	salary	department	dob
101	Nancy	4000	HR	1999-12-30
102	Lucy	6500	HR	1995-09-09
103	Mack	6000	Developer	1997-03-10
104	Charlie	9500	Tester	1998-06-24
105	Tom	3000	HR	1998-11-03

EXCEPT or MINUS

- The EXCEPT operator returns rows from the result set of the first SELECT statement that are not present in the result set of the second SELECT statement.
- It also eliminates duplicate rows.

EXCEPT / MINUS

SYNTAX :

```
SELECT column_name(s) FROM table1  
EXCEPT  
SELECT column_name(s) FROM table2  
EXCEPT  
.....;
```

SQL Questions

Question-2

- Write a SQL query to fetch all the details of the employees who are not manager from the employee table .

```

1  SELECT * FROM employee
2  EXCEPT
3  SELECT * FROM manager;

```

id	name	salary	department	dob
101	Jack	2000	HR	1997-05-19
102	Jack		HR	
104	Peter	4000	Tester	1998-07-16
106	Leo	2500	Developer	1997-10-10
107	Roger	5300	Accounts	1997-06-17
108	Mike	2000		1998-03-09
109	Paul	4800	Developer	1998-12-28
110	Hannah	2000	Accounts	1999-09-26

employee

id	name	salary	department	dob
101	Jack	2000	HR	1997-05-19
102	Jack		HR	
103	Mack	6000	Developer	1997-03-10
104	Peter	4000	Tester	1998-07-16
105	Tom	3000	HR	1998-11-03
106	Leo	2500	Developer	1997-10-10
107	Roger	5300	Accounts	1997-06-17
108	Mike	2000		1998-03-09
109	Paul	4800	Developer	1998-12-28
110	Hannah	2000	Accounts	1999-09-26

manager

id	name	salary	department	dob
101	Nancy	4000	HR	1999-12-30
102	Lucy	6500	HR	1995-09-09
103	Mack	6000	Developer	1997-03-10
104	Charlie	9500	Tester	1998-06-24
105	Tom	3000	HR	1998-11-03

INTERSECT

- The INTERSECT operator returns the common rows between the result sets of two or more SELECT statements.
- It only includes rows that appear in all the result sets.

+5 >

INTERSECT

SYNTAX :

```
SELECT column_name(s) FROM table1  
INTERSECT  
SELECT column_name(s) FROM table2  
INTERSECT  
.....;
```

Question-3

- Write a SQL query to return all the details of the employees from the employee table who are manager.

```
1 SELECT * FROM employee
2 INTERSECT
3 SELECT * FROM manager;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

id	name	salary	department	dob
103	Mack	6000	Developer	1997-03-10
105	Tom	3000	HR	1998-11-03

employee

id	name	salary	department	dob
101	Jack	2000	HR	1997-05-19
102	Jack		HR	
103	Mack	6000	Developer	1997-03-10
104	Peter	4000	Tester	1998-07-16
105	Tom	3000	HR	1998-11-03
106	Leo	2500	Developer	1997-10-10
107	Roger	5300	Accounts	1997-06-17
108	Mike	2000		1998-03-09
109	Paul	4800	Developer	1998-12-28
110	Hannah	2000	Accounts	1999-09-26

manager

id	name	salary	department	dob
101	Nancy	4000	HR	1999-12-30
102	Lucy	6500	HR	1995-09-09
103	Mack	6000	Developer	1997-03-10
104	Charlie	9500	Tester	1998-06-24
105	Tom	3000	HR	1998-11-03

```

1 SELECT * FROM employee WHERE id>104
2 INTERSECT
3 SELECT * FROM manager;

```

employee

id	name	salary	department	dob
101	Jack	2000	HR	1997-05-19
102	Jack		HR	
103	Mack	6000	Developer	1997-03-10
104	Peter	4000	Tester	1998-07-16
105	Tom	3000	HR	1998-11-03
106	Leo	2500	Developer	1997-10-10
107	Roger	5300	Accounts	1997-06-17
108	Mike	2000		1998-03-09
109	Paul	4800	Developer	1998-12-28
110	Hannah	2000	Accounts	1999-09-26

manager

id	name	salary	department	dob
101	Nancy	4000	HR	1999-12-30
102	Lucy	6500	HR	1995-09-09
103	Mack	6000	Developer	1997-03-10
104	Charlie	9500	Tester	1998-06-24
105	Tom	3000	HR	1998-11-03

Result Grid Filter Rows: Exports Wrap Cell Content:

	id	name	salary	department	dob
▶	105	Tom	3000	HR	1998-11-03

How Aggregate functions work on NULL Value (SUM, AVG, COUNT, MIN, MAX)

T₁

A	B
1 -	Null
2 -	Null
3 -	Null
4 -	Null
Null	Null

+10

Count(*)= 5 $\frac{SUM=10}{Count=4}$ SUM(A)= 10
Count(A)= 4 AVG(A)= 2.5
Count(B)= 0 MIN(A)= 1
 MAX(A)= 4

SUM(B)=
AVG(B)=
MIN(B)=
MAX(B)=
} NULL

- All aggregate functions except Count(*) ignores 'null' value
- Count of null values is defined as 0 and other functions return 'null'

SQL: Data Manipulation Language (DML)

- **Data Manipulation Language (DML)** commands are used **for accessing and manipulating the data stored in the database.**
- The DML commands are *not auto-committed* that means it can't permanently save all the changes in the database. They can be rollback.
- **Data Manipulation Language (DML)**
 - **SELECT** - to retrieve data from the database
 - **INSERT** - to insert a new row into a table
 - **UPDATE** - to update existing row in a table
 - **DELETE** - to delete a row from a table



Data Query language (DQL)

Modification of Database



1. SELECT (mostly used SQL command/query)

- **SELECT** statement is used to select a set of data from a database table. Or Simply **SELECT** statement is used to retrieve data from a database table.
 - It **returns** data in the form of a **result table**. These result tables are called **result-sets**.
- **SELECT** is also called **DQL** because it is used to **query** information from a database table
- **SELECT** statement specifies **column names**, and **FROM** specifies **table name**
- **SELECT** command is used with different **Conditions** and **CLAUSES**.

Basic Syntax:

To retrieve selected fields from the table:

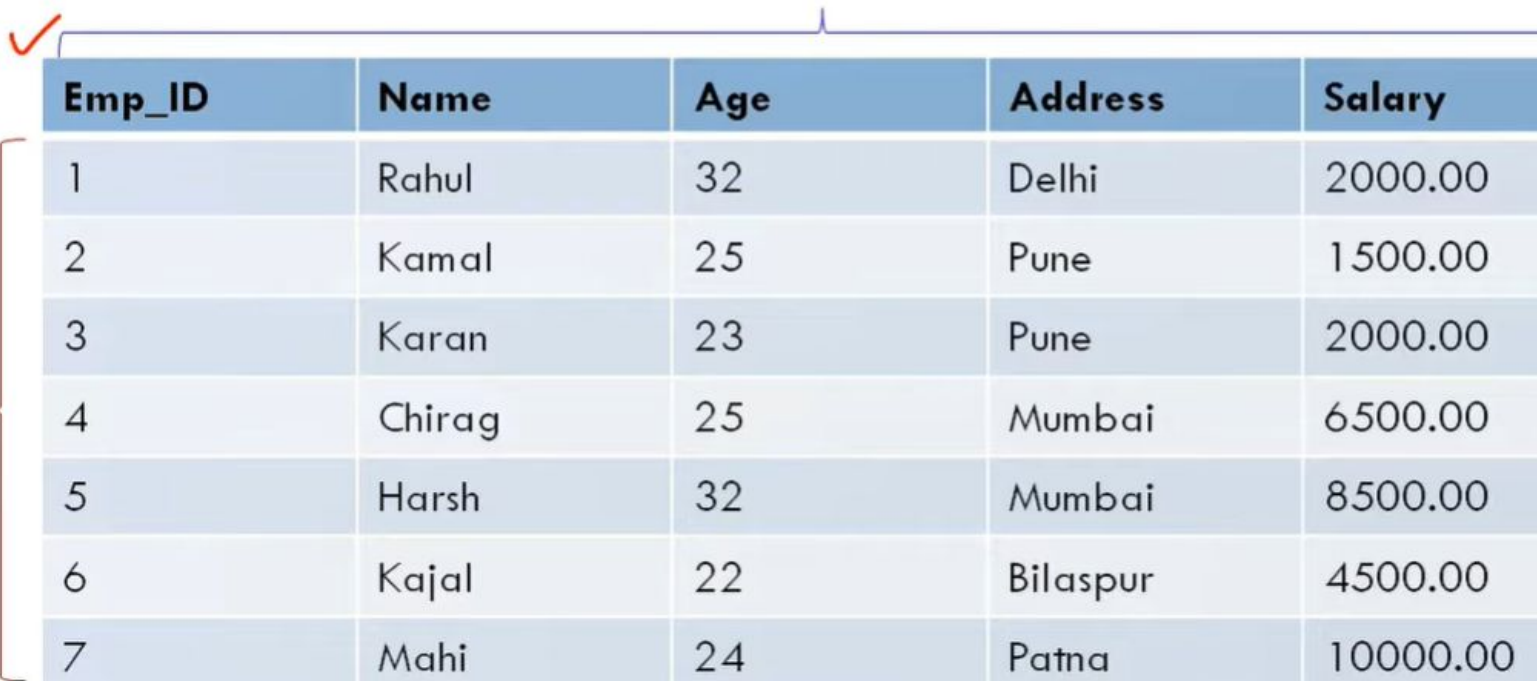
```
SELECT column1, column2, ... columnN  
FROM table_name(s);
```

To retrieve all the fields from the table:

```
SELECT *  
FROM table_name(s);
```

Example Table: **Emp**

Columns/Fields/Attributes



Emp_ID	Name	Age	Address	Salary
1	Rahul	32	Delhi	2000.00
2	Kamal	25	Pune	1500.00
3	Karan	23	Pune	2000.00
4	Chirag	25	Mumbai	6500.00
5	Harsh	32	Mumbai	8500.00
6	Kajal	22	Bilaspur	4500.00
7	Mahi	24	Patna	10000.00

Records/
Rows/
Tuples

Example 1:

To retrieve columns Emp_ID, Name from Emp table

```
SELECT Emp_ID, Name  
FROM Emp;
```



Emp_ID	Name
1	Rahul
2	Kamal
3	Karan
4	Chirag
5	Harsh
6	Kajal
7	Mahi

Example 2:

To retrieve all the fields from the Emp table

```
SELECT *  
FROM Emp;
```



Emp_ID	Name	Age	Address	Salary
1	Rahul	32	Delhi	2000.00
2	Kamal	25	Pune	1500.00
3	Karan	23	Pune	2000.00
4	Chirag	25	Mumbai	6500.00
5	Harsh	32	Mumbai	8500.00
6	Kajal	22	Bilaspur	4500.00
7	Mahi	24	Patna	10000.00



WHERE CLAUSE



- ❑ **WHERE Clause** is used to select a particular record based on a condition. It is used to filter records.
- ❑ **WHERE Clause** is used to specify a condition while fetching the data from a single table or by joining with multiple conditions
- ❑ The **WHERE** clause is not only used in the **SELECT** statement, but it is also used in the **UPDATE**, **DELETE** statement.
- ❑ Syntax: "SELECT with WHERE Clause"

SELECT column1, column2, ... columnN

FROM table_name(s)

WHERE condition;

Emp_ID	Name
4	Chirag
5	Harsh
6	Kajal
7	Mahi



To retrieve Emp_ID, Name from Emp table whose Salary is greater than 2000

SELECT Emp_ID, Name

FROM Emp

WHERE Salary > 2000;

Optional Clauses in **SELECT** statement

- ✓ ☐ [**WHERE Clause**] : It specifies which rows to retrieve by specifying **conditions**.
- ☐ [**GROUP BY Clause**] : **Groups** rows that share a **property** so that the **aggregate function** can be applied to each group.
- ☐ [**HAVING Clause**] : It **selects** among the **groups** defined by the **GROUP BY** clause by specifying **conditions**.
- ☐ [**ORDER BY**] : It specifies an order in which to return the rows.
- ☐ [**DISTINCT Clause**]: It is used to remove duplicates from results set of a **SELECT** statement. (**SELECT DISTINCT**)

SQL Operators ✓

1. SQL Arithmetic Operator
2. SQL Comparison Operators
3. SQL Logical Operators
4. SQL Special Operators

1. SQL Arithmetic Operators

Operator	Description
+	Add
-	Subtract
*	Multiply
/	Divide
%	Modulo

Example:

To retrieve Emp_ID, Name, Salary *plus 3000* from Emp table

```
SELECT Emp_ID, Name, Salary + 3000  
FROM Emp;
```

Emp_ID	Name	Salary
1	Rahul	5000.00 ✓
2	Kamal	4500.000
3	Karan	5000.00
4	Chirag	9500.00
5	Harsh	11500.00
6	Kajal	7500.00
7	Mahi	13000.00

2. SQL Comparison Operators



Operator	Description
=	Equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
<> or !=	Not equal to

Example:

To retrieve *Emp_ID*, *Name* from *Emp* table whose *Salary* is greater than 2000

```
SELECT Emp_ID, Name, Salary
FROM Emp
WHERE Salary > 2000;
```



Emp_ID	Name	Salary
4	Chirag	6500.00
5	Harsh	8500.00
6	Kajal	4500.00
7	Mahi	10000.00



3. SQL Logical Operators



Operator	Description
AND	It displays a record if <u>all</u> the conditions separated by AND are TRUE. <i><1 AND 2</i>
OR	It displays a record if <u>any</u> of the conditions separated by OR is TRUE
NOT	✓ It displays a record if the condition(s) is <u>NOT TRUE</u>. ✓ It reverses the meaning of any operator with which it is used. This is a <u>negate operator</u>. Eg: NOT EXISTS, NOT BETWEEN, NOT IN, IS NOT NULL, etc.

Example:

To retrieve all records from *Emp* table whose *salary* is greater than 2000 and *age* is less than 25

```
SELECT *  
FROM Emp  
WHERE Salary > 2000 AND Age < 25;
```



Emp_ID	Name	Age	Address	Salary
✓ 6	Kajal	22	Bilaspur	4500.00
✓ 7	Mahi	24	Patna	10000.00



4. SQL Special Operators

Operator	Description
ALL	It compares a value to all values in another value set (in subqueries).
ANY	It compares the values in the list according to the condition (in subqueries).
BETWEEN	It is used to search for values that are within a set of values (given minimum and maximum values).
EXISTS	It is used to search for the presence of a row in a specified table (in subqueries).
IN	It compares a value to that specified list value (and in subqueries).
LIKE	It compares a value to similar values using wildcard operator (% , _).
IS NULL	It checks for missing data or NULL values

Example:

To retrieve all records from Emp table with salary between 2000 and 5000

```
SELECT *  
FROM Emp  
WHERE Salary BETWEEN 2000 AND 5000;
```



Emp_ID	Name	Age	Address	Salary
1	Rahul	32	Delhi	2000.00
3	Karan	23	Pune	2000.00
6	Kajal	22	Bilaspur	4500.00

ANY Operator

SYNTAX :

```
SELECT column_name(s)  
FROM table_name  
WHERE column_name operator ANY (Subquery);
```

The *operator* must be a standard comparison operator
(=, <>, !=, >, >=, <, or <=)

SQL Questions

Questions - 1

- Write an SQL query to get all employees from the employee table whose salary is equal to any salary in the 'HR' department.

```
1 • SELECT * FROM employee;
```

Result Grid					
Filter Rows:					
Edit:    					
Export/Import: 					
id	name	salary	dob	department	
101	Jack	2000	1997-05-19	HR	
102	Jack	2000	1997-05-19	HR	
103	Mack	6000	1997-03-10	Developer	
104	Peter	4000	1998-07-16	Tester	
105	Tom	3000	1998-11-03	HR	
106	Leo	2500	1997-10-10	Developer	
107	Roger	5300	1997-06-17	Accounts	
108	Mike	2000	1998-03-09	HR	
109	Paul	4800	1998-12-28	Developer	
110	Hannah	2000	1999-09-26	Accounts	

Answer :

SELECT *

FROM employee

WHERE salary = **ANY** (

SELECT salary

FROM employee

WHERE department = 'HR'

):

Answer :

SELECT *

FROM employee

WHERE salary = **ANY** (2000,3000);

```

1 • SELECT *
2   FROM employee
3   WHERE salary = ANY (
4     SELECT salary
5     FROM employee
6     WHERE department = 'HR'
7   );

```

employee

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Contents:

id	name	salary	dob	department
101	Jack	2000	1997-05-19	HR
105	Tom	3000	1998-11-03	HR
108	Mike	2000	1998-03-09	HR
110	Hannah	2000	1999-09-26	Accounts

id	name	salary	dob	department
101	Jack	2000	1997-05-19	HR
102	Jack			HR
103	Mack	6000	1997-03-10	Developer
104	Peter	4000	1998-07-16	Tester
105	Tom	3000	1998-11-03	HR
106	Leo	2500	1997-10-10	Developer
107	Roger	5300	1997-06-17	Accounts
108	Mike	2000	1998-03-09	HR
109	Paul	4800	1998-12-28	Developer
110	Hannah	2000	1999-09-26	Accounts

ALL Operator

- The ALL operator is used in combination with comparison operators to compare a value with a set of values returned by a subquery.
(*Multiple-row subquery*)
- It returns true if the comparison holds true for all values in the set.
- The ALL operator is often used with the WHERE or HAVING clause.

SYNTAX :

```
SELECT column_name(s)
FROM table_name
WHERE column_name operator ALL (Subquery);
```

The *operator* must be a standard comparison operator
(=, <>, !=, >, >=, <, or <=)

SQL Questions

Questions - 2

- **Write an SQL query to retrieve all employees whose salary is greater than the salary of all employees in the Accounts department.**

ANY Operator

Answer :

SELECT *

FROM employee

WHERE salary > **ALL** (

SELECT salary

FROM employee

WHERE department = 'Accounts ')

);

(5300,2000)

```

1 • SELECT *
2   FROM employee
3   WHERE salary > ALL (
4     SELECT salary
5     FROM employee
6     WHERE department = 'Accounts'
7   );

```

employee

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

id	name	salary	dob	department
103	Mack	6000	1997-03-10	Developer

id	name	salary	dob	department
101	Jack	2000	1997-05-19	HR
102	Jack			HR
103	Mack	6000	1997-03-10	Developer
104	Peter	4000	1998-07-16	Tester
105	Tom	3000	1998-11-03	HR
106	Leo	2500	1997-10-10	Developer
107	Roger	5300	1997-06-17	Accounts
108	Mike	2000	1998-03-09	
109	Paul	4800	1998-12-28	Developer

EXISTS Operator

- The EXISTS operator is used to test for the existence of any record in a subquery.
- It returns TRUE if the subquery returns one or more records.
- It is useful when you want to conditionally retrieve data based on the existence of related records in same or another table.
- EXISTS operator generally used with Correlated Subqueries.
- The EXISTS operator is typically used in combination with the WHERE clause in SQL

SQL Questions

Questions - 3

- **Write an SQL query to retrieve the names of customers who have placed at least one order.**

```
1 • SELECT * FROM customers;
```

	id	name	email	address
▶	C1	John Doe	john.doe@example.com	123 Main St
	C2	Jane Smith	jane.smith@example.com	456 Elm St
	C3	Michael Johnson	michael.johnson@exam...	789 Oak St
	C4	Emily Davis	emily.davis@example.com	987 Pine St
	C5	David Wilson	david.wilson@example.c...	654 Maple St
*	NULL	NULL	NULL	NULL

```
1 • SELECT * FROM orders;
```

	id	order_date	total_amount	customer_id
▶	1	2023-07-10	150.25	C1
	2	2023-07-11	75.50	C1
	3	2023-07-12	200.00	C2
	4	2023-07-12	350.75	C3
	5	2023-07-13	100.00	C3
	6	2023-07-13	125.30	C5
	7	2023-07-14	50.75	C1
*	NULL	NULL	NULL	NULL

```
SELECT name
FROM customers
WHERE EXISTS (
    SELECT *
    FROM orders
    WHERE orders.customer_id = customers.id
);
```

Id	Name	Email	Address
C1	John Doe	john.doe@example.com	123 Main St
C2	Jane Smith	jane.smith@example.com	456 Elm St
C3	Michael Johnson	michael.johnson@example.com	789 Oak St
C4	Emily Davis	emily.davis@example.com	987 Pine St
C5	David Wilson	david.wilson@example.com	654 Maple St

Customers

Id	Order_date	Total_amount	Customer_id
1	2023-07-10	150.25	C1
2	2023-07-11	75.50	C1
3	2023-07-12	200.00	C2
4	2023-07-12	350.75	C3
5	2023-07-13	100.00	C3
6	2023-07-13	125.30	C5
7	2023-07-14	50.75	C1

Orders

```
1 • SELECT name
2   FROM customers
3  WHERE EXISTS (
4      SELECT *
5      FROM orders
6      WHERE orders.customer_id = customers.id
7  );
```

<	
Result Grid	Filter Rows:
name	Exports: Wrap Cell Contents:
▶ John Doe	
Jane Smith	
Michael Johnson	
David Wilson	

IN Operator

Answer :

```
SELECT name
FROM customers
WHERE name IN (
    SELECT name
    FROM orders
    WHERE orders.customer_id = customers.id
);
```

2. INSERT



- The **INSERT INTO** statement is used to insert new records in a table.
- It is possible to write the **INSERT INTO** statement in two ways.

Syntax 1: Specify both the column names and the values to be inserted

```
INSERT INTO table_name(column1, column2, column3,...columnN)  
VALUES (value1, value2, value3,...valueN);
```

Syntax 2: Specify only values to be inserted. But needed to remember the column order

```
INSERT INTO table_name  
VALUES (value1, value2, value3,...valueN);
```

Example: INSERT INTO



✓ **INSERT INTO** Emp (Emp_ID, Name, Age, Address, Salary) **VALUES** (1, 'Rahul', 32, 'Delhi', 2000);

➤ **INSERT INTO** Emp (Emp_ID, Name, Age, Address, Salary) **VALUES** (2, 'Kamal', 25, 'Pune', 1500);

➤ **INSERT INTO** Emp (Emp_ID, Name, Age, Address, Salary) **VALUES** (3, 'Karan', 23, 'Pune', 2000);

➤ **INSERT INTO** Emp **VALUES** (4, 'Chirag', 25, 'Mumbai', 6500);

➤ **INSERT INTO** Emp **VALUES** (5, 'Harsh', 32, 'Mumbai', 8500);

➤ **INSERT INTO** Emp **VALUES** (6, 'Kajal', 22, 'Bilaspur', 4500);

➤ **INSERT INTO** Emp **VALUES** (7, 'Mahi', 24, 'Patna', 10000);

1st way

2nd way

✓ **To Check:**

SELECT *
FROM Emp;



Emp_ID	Name	Age	Address	Salary
1	Rahul	32	Delhi	2000.00
2	Kamal	25	Pune	1500.00
3	Karan	23	Pune	2000.00
4	Chirag	25	Mumbai	6500.00
5	Harsh	32	Mumbai	8500.00
6	Kajal	22	Bilaspur	4500.00
7	Mahi	24	Patna	10000.00



3. UPDATE

- The **UPDATE** statement is used to modify the existing records in a table. ✓
- ✓ **Note:** Always use the WHERE clause with the UPDATE statement to update the selected rows, otherwise all the rows would be affected.

Syntax:

UPDATE *table_name*

SET *column1 = value1, column2 = value2, columnN = valueN*

✓ **WHERE** *condition;*

Example: UPDATE

Example: To update the address of Emp_ID: 3 to **Chennai**

```
✓ UPDATE Emp
  SET Address='Chennai'
  WHERE Emp_ID=3;
```

✓ To Check:

```
SELECT *
FROM Emp;
```



Emp_ID	Name	Age	Address	Salary
1	Rahul	32	Delhi	2000.00
2	Kamal	25	Pune	1500.00
3	Karan	23	Chennai	2000.00
4	Chirag	25	Mumbai	6500.00
5	Harsh	32	Mumbai	8500.00
6	Kajal	22	Bilaspur	4500.00
7	Mahi	24	Patna	10000.00

4. DELETE

□ The **DELETE** statement is used to delete existing records in a table. ✓

✓ □ **Note:** Always use the WHERE clause with a DELETE statement to delete the selected rows, otherwise all the records would be deleted.

Syntax:

```
DELETE FROM table_name  
WHERE condition;
```

7-MINUT

Example: DELETE

Example:

To delete the employee records of **Pune** location

```
DELETE FROM Emp  
WHERE Address='Pune'; ✓
```

✓ **To Check:**

```
SELECT *  
FROM Emp;
```



Emp_ID	Name	Age	Address	Salary
1	Rahul	32	Delhi	2000.00
4	Chirag	25	Mumbai	6500.00
5	Harsh	32	Mumbai	8500.00
6	Kajal	22	Bilaspur	4500.00
7	Mahi	24	Patna	10000.00

Integrity Constraints over Relation



- **Integrity constraints** are used to ensure accuracy and consistency of the data in a relational database.
- **Integrity constraints** are **set of rules** that the database is not permitted to violate.
- **Constraints** may apply to each **attribute** or they may apply to **relationship between tables**.
- **Integrity constraints** ensure that changes (update, deletion, insertion) made in the database by authorized users do not result in a loss of data consistency. Thus, ***integrity constraints guard against accidental damage to the database.***
 - **Example** - A blood group must be 'A' or 'B' or 'AB' or 'O' only (can not any other values else).

TYPES OF INTEGRITY CONSTRAINT

1. **Domain Constraint** / A
2. **Entity Integrity Constraint** - A
3. **Referential Integrity Constraint** / R-R
4. **Key Constraints** / A

1. Domain Constraints



- **Domain constraints** defines the domain or the valid set of values for an attribute.
- The **data type of domain** includes string, character, integer, time, date, currency, etc. *The value of the attribute must be available in the corresponding domain.*

age → +ve int

STUDENT_ID	NAME	SEMESTER	AGE
101	Manish	1st	18
102	Rohit	3rd	19
103	Badal	5th	20
104	Amit	7th	A

Not allowed. Because AGE is an integer value

100 — "A"

Student table unique check age < 18

default "A"

NOT NULL

name	rollno	age	Section
Sai	1	17	A
Shiva	2	10	A
NIKE Ramesh	3	20	A

2. Entity Integrity Constraints



- The **entity integrity constraint** states that primary key value can't be null.
- This is because the primary key value is used to identify individual rows in relation and if the primary key has a null value, then we can't identify those rows.
- A table can contain a null value other than the primary key field.

EMP_ID	EMP_NAME	SALARY
111	Mohan	20000
112	Rohan	30000
113	Sohan	35000
	Logan	20000



Not allowed as Primary Key can't contain NULL value

3. Referential Integrity Constraints



- A **referential integrity constraint** is specified between two tables.
- **Referential Integrity constraint** is enforced when a foreign key references the primary key of a table.
- In the **Referential integrity constraints**, if a foreign key in Table 1 refers to the Primary Key of Table 2, then **either** every value of the foreign Key in Table 1 must be available in primary key value of Table 2 **or** it must be null.
- The **rules** are:
 - ▣ You can't **delete** a record from a *primary table* if matching records exist in a related table.
 - ▣ You can't **change** a primary key value in the *primary table* if that record has related records.
 - ▣ You can't **insert** a value in the foreign key field of the *related table* that doesn't exist in the primary key of the *primary table*.
 - ▣ However, you can enter a **Null** value in the foreign key, specifying that the records are unrelated.

EMP (Table 1): Related /Referencing Table

PK

<u>EMP_ID</u>	EMP_NAME	AGE	DEP_NO
111	Mohan	21	1
112	Rohan	33	2
113	Sohan	27	3
114	Logan	25	5

Foreign Key

Relationships

Primary Key

DEP (Table 2): Primary/Referenced Table

<u>DEP_NO</u>	LOCATION
1	Mumbai
2	Delhi
3	Noida

Not allowed as DEP_NO 5 is not defined as a Primary Key of Table 2 and In Table 1, DEP_NO is a foreign key defined



4. Key Constraints



- An entity set can have multiple keys or candidate keys (minimal superkey), but out of which one key will be the primary key.
- **Key constraint** specifies that in any relation-
 - ▣ All the values of primary key must be **unique**.
 - ▣ The value of primary key must **not be null**.

pk

<u>STUDENT_ID</u>	NAME	SEMESTER	AGE
101	Manish	1st	18
✓ 102	Rohit	3rd	19
103	Badal	5th	20
- 102	Amit	7th	21

Not allowed. Because all rows must be **unique**

Triggers in SQL

A trigger is a statement that the system executes automatically as a side effect of a modification to the database.

Write a PL-SQL code which will insert the data automatically into 'Backup' table whenever data is deleted from 'Main' table.

ID	Salary
1	10
2	20
3	30
4	40

Main

Backup	
ID	Salary

Use Triggers

```
Create or Replace trigger t1  
before delete on Main  
for each row  
begin  
insert into Backup values(:old.id, :old.Salary);  
end;
```

```
create table main  
(id int,  
salary int);
```

```
insert into main values(1,10000);  
insert into main values(2,20000);
```

```
create table Backup  
( id int,  
salary int);
```

CODE

[Oracle FreeSQL](#)

```
Create table main (id int, salary int);  
Create table backup(id int, salary int);
```

```
Insert into main values(1,20000);  
Insert into main values(2,30000);
```

```
Create or replace trigger t1  
Before delete on main  
For each row  
Begin  
Insert into backup values(:old.id,:old.salary);  
End;  
Delete from main where id=2;
```

Database Security in DBMS

- Database Security ensures confidentiality, integrity, and availability of data.
- Protects data from unauthorized access, misuse, or corruption.
- Example: Banking system security.

Q. What is data security & authorization? full explanation.

Ans → In DBMS, ensuring data security and proper authorization is crucial to protect sensitive information and maintain data integrity.

② User Authorization :- Authorization determines what authenticated users are permitted to do within the database.

Example:- User 'ANKIT' have privileges to Connect, Insert, Select, Alter data in the 'CUSTOMERS' table but not to delete.

Now, ANKIT can do only provided privileges in database he can't delete anything on database.

Command Prompt - sqlplus

Connected to:
Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64
bit Production

SQL> show user
USER is "SYSTEM"

SQL> create user akhil identified by akhil2klc;

User created.

SQL> commit;

Commit complete.

SQL>

intersect.fbr
union.wmv
ChatCode

minus.fbr
selection.w...
Akhilesh -
Chrome

Command Prompt - sqlplus

Enter user-name: akhil

Enter password:

ERROR:

ORA-01045: user AKHIL lacks CREATE SESSION privilege; logon
denied

Enter user-name: akhil

Enter password:

ERROR:

ORA-01045: user AKHIL lacks CREATE SESSION privilege; logon
denied

Enter user-name:

```
SQL> grant connect to akhil;
```

```
Grant succeeded.
```

```
SQL> commit;
```

```
Commit complete.
```

```
SQL>
```



Command Prompt - sqlplus

```
Enter user-name: akhil
```

```
Enter password:
```

```
ERROR:
```

```
ORA-01045: user AKHIL lacks CREATE SESSION privilege; logon  
denied
```

```
Enter user-name: akhil
```

```
Enter password:
```

```
Connected to:
```

```
Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64b  
it Production
```

```
SQL> |
```

